

专题：里程计漂移与矫正

这是第二节课没讲透、第三节课一上来就要用的地基。

第二节做出了地图，也让你见过 TF 里多出一条 `map → odom` ——当时多半只记住「有这条边」。本专题专门回答：

1) 轮子为什么会漂；2) 为什么不能直接改 `/odom`；3) 矫正量怎么算出来；4) 数值多大算正常。

第三节导航主线（加载地图 → 初始定位 → 行走中矫正 → 规划跟线）建立在这些答案之上。主课讲义见 `docs/10.3.3_Nav2导航系统与路径规划.md`。

文中曲线来自 Gazebo 巡航一圈的真实录制，复现命令见第 5 节。

0. 和第二节、第三节怎么衔接

课次	当时你看到什么	本专题补什么
第 2 课	SLAM 发 <code>/map</code> 和 <code>map → odom</code> ；墙被「刷」出来	为什么需要 <code>map → odom</code> ，而不是只有 <code>odom</code>
第 3 课	地图冻结；改由 AMCL 发同一条 <code>map → odom</code>	发布者换了， 账本机制不变 ；导航靠它边走边纠偏

读完本专题，应能用一句话回答第三节开场必问：

里程计负责连续，激光负责正确，`map → odom` 是两者的差值账本。

1. 问题：机器人凭什么知道自己在哪？

机器人估计自身位置只有两类信息来源：

1. **航迹推算 (dead reckoning)**：读轮子转了多少圈，按运动学积分出"我应该走到了哪"。对应 `/odom` 话题和 TF `odom → base_footprint`。
2. **环境观测**：拿传感器（本项目是激光雷达）看周围，和已知地图比对，"照镜子"确认自己的绝对位置。

航迹推算快而平滑但会漂，环境观测准但慢且会跳。漂移矫正的本质，就是把两者的长处拼起来。

2. 漂移从哪来

里程计的每一步误差都很小，但它是积分出来的——误差只进不出，越走越歪：

误差源	机理	麦轮上是否更严重
轮子打滑	轮子转了、车没动（或动少了），编码器照常计数	严重 。麦轮横移靠辊子摩擦分解速度，本质上是"受控打滑"
轮径/轮距标定误差	运动学参数与实物不一致，每米固定比例偏差	一样
地面不平/碰撞	车身姿态突变，编码器无感	一样
角度误差放大	朝向偏 1°，直行 5m 后横向偏差约 9cm	严重 。原地自转是四轮同时打滑

经验数量级：编码器里程计漂移约为行驶距离的 1%~5%；麦轮车原地旋转后的角度误差尤其明显。**角度漂移比平移漂移危害大**——它会把后续所有位移都投影到错误的方向上。

本项目两个后端的"漂移现实"：

- Gazebo 后端：**`MecanumDrive` 插件的里程计由轮速运动学积分得到，物理引擎里轮子与地面真实打滑，所以漂移是**物理产生**的，无需人为注入。
- 轻量后端：**`simulator_node` 的里程计默认直接输出真值（`odom_xy_noise_std: 0.0`），漂移为零——这是刻意设计，方便第二课把 SLAM 问题和里程计问题分开教学。想在轻量后端也看到漂移，运行期打开两个噪声参数即可（无需改代码、无需重启）：

```
# 每次里程计更新叠加标准差 1cm 的平移高斯噪声、约 0.3° 的角度噪声
ros2 param set /mecamind_simulator_node odom_xy_noise_std 0.01
ros2 param set /mecamind_simulator_node odom_yaw_noise_std 0.005
```

打开后再用第 6 节的方法观察 `map → odom`，就能在轻量后端复现出和 Gazebo 类似的阶梯曲线。

3. 为什么不直接"修" /odom：TF 二段式设计

最容易想到的方案是：定位算出准确位姿后，直接覆盖 `/odom`。ROS 不这么做，而是把位姿拆成两段 TF 相乘：



机器人在 `map` 系下的真实位姿 = `(map→odom) × (odom→base_footprint)`。

这样拆的理由是**两类消费者的需求矛盾**：

消费者	需要什么	用哪段
控制器（DWB, 10 Hz）	位姿 连续 ，绝不能跳变，否则速度指令抖动	<code>odom → base_footprint</code>
规划器 / 任务层	位姿 准确 ，慢点、偶尔跳一下没关系	完整链 <code>map → base_footprint</code>

如果直接覆盖 `/odom`，每次定位修正都会让控制器看到位姿突跳——车会猛打方向。拆成两段后：**里程计永远连续，修正只发生在 `map → odom` 这一段**，控制器完全感知不到。这也正是第三课局部代价地图挂在 `odom` 系的原因。

一个直观类比：里程计是你闭着眼睛数步子，`map → odom` 是你偶尔睁眼看一下路牌后对"步数起点"的整体平移——你不需要否认已经数过的步子，只需要挪一下参考点。

4. 矫正原理：漂移是怎么被"算"掉的

先给分工表，再逐层拆开"算"的过程。

	第二课（建图）	第三课（导航）
<code>map → odom</code> 的发布者	<code>slam_toolbox</code>	<code>amcl</code>
求真实位姿的方法	扫描匹配（当前激光帧 vs 正在生长的地图）	粒子滤波（当前激光帧 vs 冻结的静态地图）
修正频率	每次扫描匹配成功	车移动 <code>update_min_d: 0.15 m</code> 或转 <code>update_min_a: 0.15 rad</code> 才更新一次

4.1 核心公式：修正量不是"测"出来的，是"减"出来的

任何时刻系统手里有两个位姿：

- **里程计位姿** $T_{\{odom \ to \ base\}}$ ：轮速积分的结果，含漂移；
- **真实位姿** $T_{\{map \ to \ base\}}$ ：激光和地图比对估计出来的（怎么估计见 4.2/4.3）。

`map → odom` 不需要额外传感器，它就是这两者的差：

$$T_{\{map \ to \ odom\}} = T_{\{map \ to \ base\}} \cdot \left(T_{\{odom \ to \ base\}} \right)^{-1}$$

一个数值例子（忽略角度，便于口算）：

1. 机器人真实走到了 `map` 系的 $(2.00, \ 1.00)$ ，激光匹配也估计出 $(2.00, \ 1.00)$ 。
2. 里程计因打滑只积到 $(1.90, \ 0.95)$ 。
3. 于是 $T_{\{map \ to \ odom\}} = (2.00-1.90, \ 1.00-0.95) = (0.10, \ 0.05)$ ——正是第 5 节曲线里那种几厘米的量。
4. 下游任何节点要 `map` 系位姿时，TF 自动做加法： $(1.90, 0.95) + (0.10, 0.05) = (2.00, 1.00)$ ，漂移被"透明地"补掉了。

注意两点：

- 有角度时不是简单加减而是位姿矩阵相乘，但思想一样："真实位姿 = 修正量 ◦ 里程计位姿"，解出修正量。
- 里程计**从未被改写**。它继续按自己的节奏漂，每次"结算"只更新 `map → odom` 这一条边。

"结算"到底指哪几步？ 就是下面这条固定流水线，建图和导航都一样，唯一区别是第 ② 步的算法不同：

步骤	做什么	输入 → 输出
① 记账	从 TF 读当前里程计位姿	→ $T_{\text{odom} \rightarrow \text{base}}$ (比如 $(1.90, 0.95)$ ，含漂移)
② 对表	拿当前激光帧和地图比对，估计真实位姿（建图用扫描匹配 § 4.2，导航用粒子滤波 § 4.3）	激光帧 + 地图 → $T_{\text{map} \rightarrow \text{base}}$ (比如 $(2.00, 1.00)$)
③ 算差	代入本节公式相减（有角度时求逆再相乘）	→ $T_{\text{map} \rightarrow \text{odom}} = (0.10, 0.05)$
④ 发布	把这个差作为新的 <code>map → odom</code> TF 广播出去，覆盖旧值	旧值 $(0.08, 0.04)$ → 新值 $(0.10, 0.05)$ ，第 5 节曲线跳一格

再连着结两次账，看漂移是怎么被"滚动吸收"的：

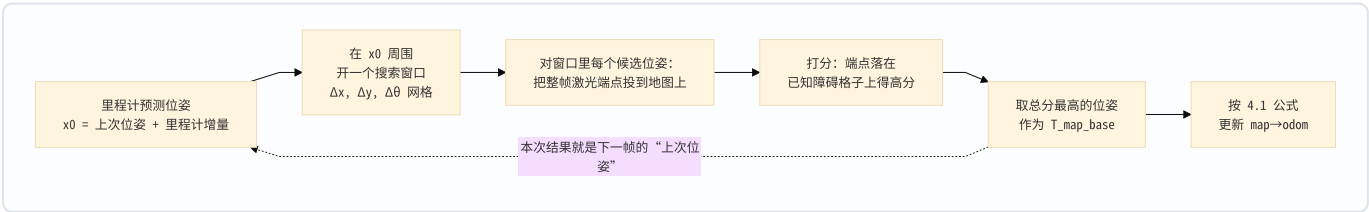
时刻	里程计 ①	激光估计 ②	map → odom ③④	说明
第 1 次结算	$(1.90, 0.95)$	$(2.00, 1.00)$	$(0.10, 0.05)$	吸收了出发以来的全部漂移
两次结算之间	继续积分、继续漂	不算	保持 $(0.10, 0.05)$ 不变	阶梯的"平台"；map 系位姿 = 里程计 + 旧修正量，仍然连续
第 2 次结算	$(2.35, 1.10)$	$(2.48, 1.17)$	$(0.13, 0.07)$	新漂出来的 $(0.03, 0.02)$ 被并入账本；曲线跳一格（阶梯的"竖边"）

所以"结算"不是去修正里程计的历史数据，而是每次都拿最新的两个位姿重新做一次减法，用结果覆盖 `map → odom`。里程计这本"糊涂账"一个字没改，账本 `map → odom` 却始终记着"糊涂账和真账的最新差额"——这就是第 3 节"挪参考点，不改步数"的数学表达。

矫正问题于是归约成一个问题：第 ② 步里，怎么从激光帧估计出 $T_{\text{map} \rightarrow \text{base}}$ ？ 建图课和导航课给了两种答案。

4.2 答案一（建图课）：扫描匹配——把激光帧"套"到地图上

slam_toolbox 用相关性扫描匹配 (correlative scan matching)，本质是一场穷举打分：



先回答图里的第一个疑问："上次位姿"是哪来的？ 这是一个递推链，往回追一定能追到头：

- 第 N 帧用的"上次位姿"，就是第 N-1 帧扫描匹配算出来的结果（图中那条虚线回边）；
- 一直追到第 1 帧——它没有"上一帧"，用的是时刻 0 的初始位姿；

- 时刻 0 的初始位姿哪来的？**建图时不要求，直接规定**：SLAM 启动瞬间，机器人当前的落脚点被定义为 map 系原点 $(0,0,0)$ ，整张地图都是相对这个点画出来的。我们项目里 `named_goals.yaml` 写着 `initial_pose: {x: 0, y: 0}`、bedroom 是 $(2.5, 0.6)$ ，意思就是"出生点即原点、卧室在出生点右 2.5 m 上 0.6 m"——这些坐标全是第二课建图时那个"规定"的产物。

所以建图阶段**不存在"初始定位"问题**（原点是定义出来的，不是测出来的）；而第三课导航时地图已经建好、原点已经固定，机器人可能被放在图上任何地方，第 1 帧的"上次位姿"就必须由人给（`/initialpose` 或 RViz 2D Pose Estimate）——这正是两课在初始位姿上的本质区别，也是初始位姿给错后果严重的原因：递推链的起点错了，后面每一帧都在错误位置附近做局部搜索，永远追不回来（见第 7 节"修正量突然跳到几米"）。

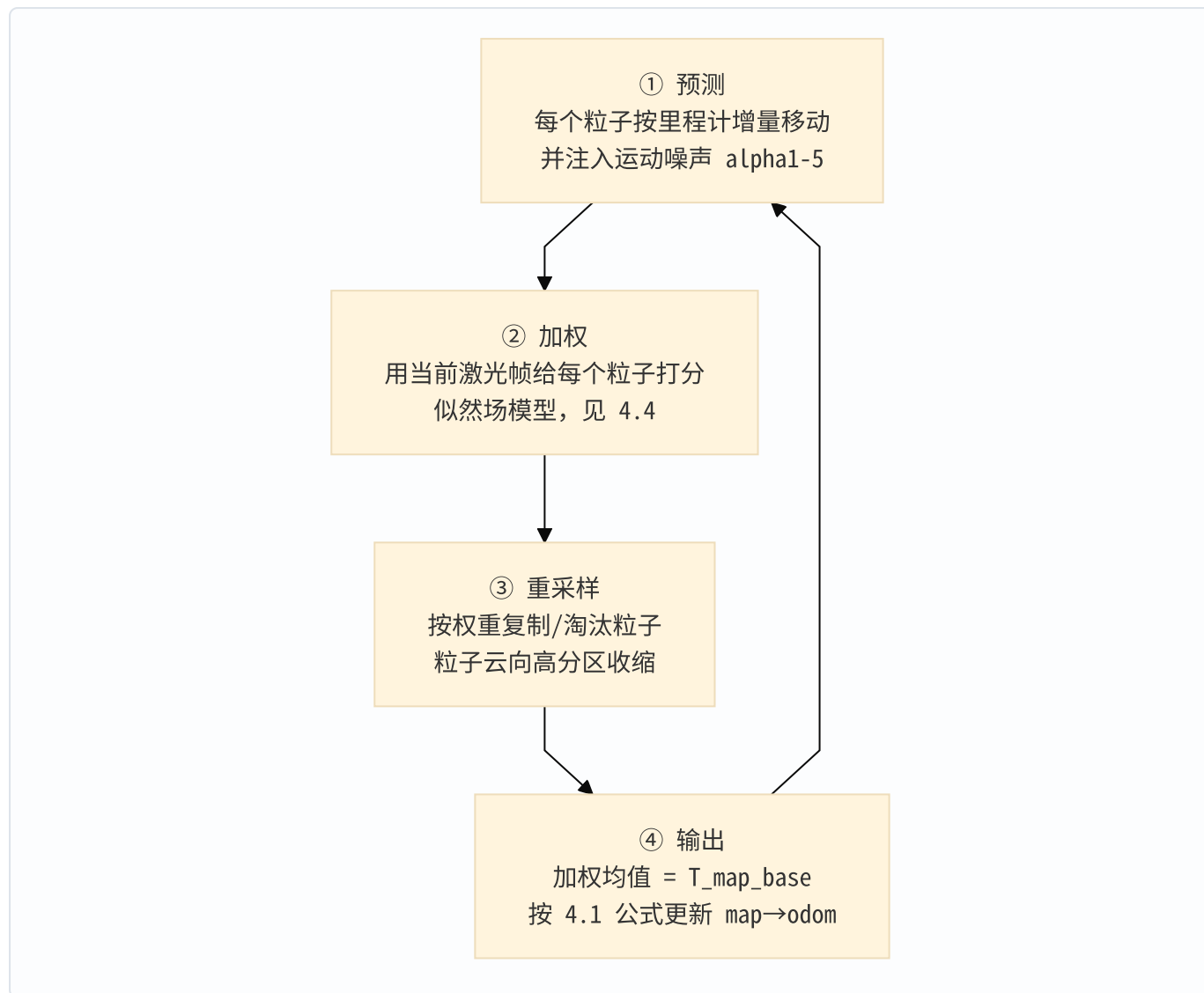
打分的直觉：**如果猜的位姿是对的，激光点应该恰好"贴"在地图的墙上**。位姿猜偏 5 cm，所有激光点整体偏 5 cm，大量点落到空白区，总分立刻掉下来。分数关于位姿偏移非常敏感，这就是激光矫正能到厘米级的原因。

三个关键设计：

- **里程计缩小搜索范围**：不用全图搜，只在里程计预测点附近开一个小窗口（漂移在两帧之间最多几厘米）。这就是"短期信里程计"的落点——它把一个全局搜索问题变成局部搜索。
- **由粗到细**：先用低分辨率网格粗搜，锁定小区域后再用高分辨率细搜，把穷举的计算量压下来。
- **匹配失败要能认怂**：地图特征太少（长走廊、空旷区）时最高分和次高分接近，说明"套哪都差不多"，此时宁可不更新、继续信里程计，也不能强行套一个错的。

4.3 答案二（导航课）：粒子滤波——用一群"猜测"逼近真实位姿

AMCL 不输出单一位姿，而是维护 500~2000 个带权重的**粒子**（每个粒子 = 一个位姿猜测 + 一个"这个猜测有多可信"的权重），循环四步：



逐步拆开：

① **预测（运动模型）**：里程计说"前进了 0.15 m"，每个粒子就前进约 0.15 m——但**每个粒子加的噪声不同**。噪声大小由 $\alpha_1 \sim \alpha_5$ （本项目全部 0.2）控制： α 越大表示"越不信任里程计"，粒子云散得越快。这一步是刻意把里程计的不确定性**显式建模**出来：漂移不可预测，但"漂移大概有多大"可以统计。

- 本项目用 `OmniMotionModel`（含 α_5 横移噪声项）：麦轮能横移，横移打滑最狠，必须给横向单独留噪声通道。差分模型没有这个通道，会把横移观测硬解释成"前进+旋转"的组合，几步之内定位就崩——这就是第三课强调运动模型必须选对的原理层原因。

② **加权（观测模型）**：拿真实激光帧问每个粒子："如果机器人真在这，激光应该是什么样？和实际看到的像不像？"像的权重高，不像的权重趋近零。打分细节见 4.4。

③ **重采样**：按权重抽签复制下一代粒子——高分粒子被复制多份，低分粒子被淘汰。效果是粒子云**向解释得通激光的区域收缩**。第三课 LAB 里"机器人走几米后粒子云明显收缩"就是这一步的宏观表现。

④ **输出**：所有粒子的加权均值就是 $T_{map \rightarrow base}$ 的最优估计，代入 4.1 的公式， $map \rightarrow odom$ 更新一格。

为什么用一群粒子而不是一个估计值？因为**漂移矫正本质是多假设问题**：激光在对称环境里可能"套"到多个位置（两个长得一样的房间），单点估计一旦选错就永远错下去；粒子云可以同时押注多个假设，等机器人走到有区分度的地方，错误假设的权重自然归零。

4.4 似然场：一个粒子的分数怎么算

AMCL 默认的观测模型是**似然场 (likelihood field)**。预处理阶段对静态地图做一次距离变换：每个格子存"离最近障碍物多远"。打分时：

1. 假设机器人在粒子位姿处，把激光每根光束的**端点**投影到地图坐标。
2. 查该端点格子的"最近障碍距离" d 。
3. 单束得分 $\propto \exp(-d^2 / 2\sigma^2)$ ：端点正好落在墙上 $d=0$ 得满分；离墙越远分越低。
4. 全帧光束得分连乘（对数域求和），得到该粒子权重。

似然场的巧处在于它是一个平滑的评分面：粒子位姿稍微偏一点，分数平滑地降一点（而不是从"命中"跳到"未命中"），这让粒子云能沿着分数梯度稳定收敛，抗激光噪声的能力也强——第二课我们给激光加的 0.01 m 高斯噪声，在这套评分下几乎不影响定位。

4.5 为什么修正是"阶梯状"且低频

第 5 节实测曲线的阶梯形态由三个设计共同决定：

1. **更新门槛**：车移动不足 `update_min_d: 0.15` m 且转动不足 `update_min_a: 0.15` rad 时，AMCL 干脆不做滤波循环——车没怎么动，里程计还没漂出新账，算了也白算。于是修正量在两次更新之间保持常数（阶梯的"平台"）。
2. **每次更新一步到位**：滤波循环执行时，粒子云收缩后的均值与上一次可能差几厘米，`map → odom` 直接跳过去（阶梯的"竖边"）。不做插值平滑——因为消费 `map` 系的都是慢速决策者（规划器），跳变无害；控制器在 `odom` 系，根本看不见这个跳变。
3. **低频是优点不是缺陷**：矫正做得越频繁，CPU 花得越多，收益却递减（0.15 m 内里程计漂移通常不到 1 cm）。"里程计管高频、激光管低频"是刻意的算力分配。

想亲手验证阈值的作用：把 `mecamind_nav2_params.yaml` 里 `update_min_d` 从 0.15 改成 0.5，重跑第 6 节的录制——阶梯会变宽、单次跳变变大，矫正更省 CPU 但更"顿"。反过来想，如果把矫正频率推到极限（每帧激光都结算，`map → odom` 以几十 Hz 跳动）会怎样？`map` 系位姿会变成高频抖动的信号，任何不小心用了 `map` 系的控制环节都会跟着抖——这又回到第 3 节两类消费者的矛盾：跳变必须被限制在低频、只给慢速决策者消费。

4.6 一句话总结

里程计负责"连续"，激光负责"正确"，`map → odom` 是两者的差值账本——扫描匹配（建图）或粒子滤波（导航）每隔一小段路把账结算一次。它不为零、且随时间波动，是系统在正常工作的标志；恒等于零反而说明没有任何矫正正在发生。

4.7 接到第三节导航主线

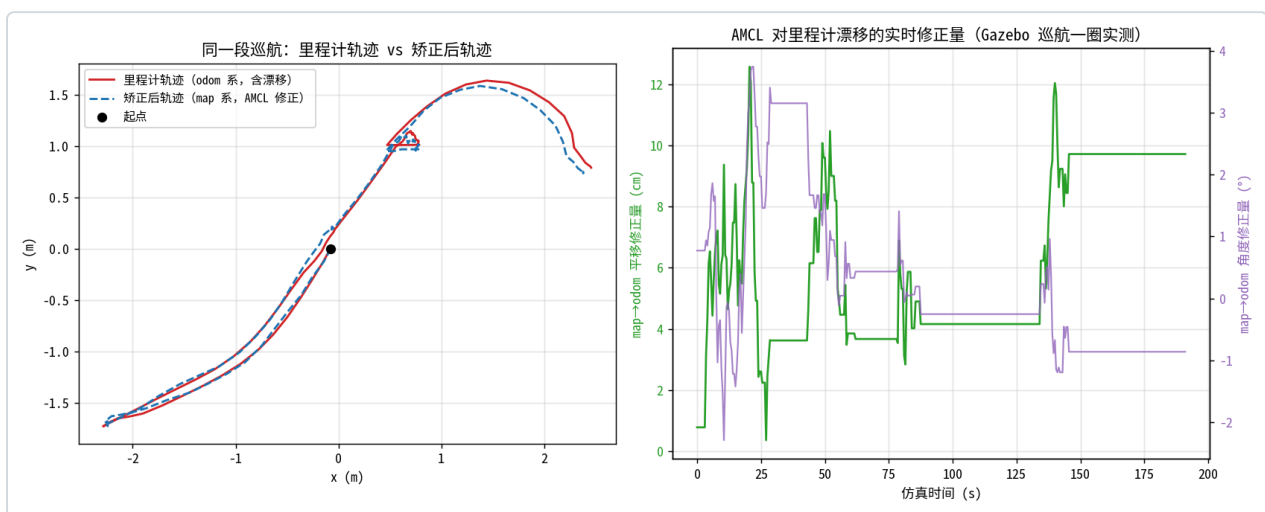
本专题只解决「位置怎么保持正确」。第三节在此之上继续：

1. 加载地图：map_server 读冻结的 PGM/YAML（第二节产物）。
2. 初始定位：/initialpose + AMCL，回答「一开始我在地图哪」。
3. 行走矫正：就是本专题的 AMCL 更新 map → odom。
4. 走到目标：代价地图上规划 + 局部跟线（Nav2 的 planner / controller）。

不要颠倒顺序：先懂账本，再背节点名。主课正文按这四步展开：docs/10.3.3_Nav2导航系统与路径规划.md。

5. 实测：巡航一圈，漂移和矫正长什么样

实验方法：Gazebo 后端启动导航栈，机器人巡航一圈（hall_entry → living_room → bedroom），期间用 scripts/record_drift.py 以 0.5s 间隔同时记录 map → odom 与 odom → base_footprint 两段 TF，共 315 个采样点。



里程计漂移与 AMCL 矫正实测

左图（轨迹对比）：红色实线是纯里程计轨迹（odom 系），蓝色虚线是 AMCL 矫正后的轨迹（map 系）。两条线整体形状一致，但存在肉眼可见的分离——分离量就是漂移。走廊直行段两线几乎重合，转弯和横移后分离变大（麦轮打滑集中发生在这些动作里）。

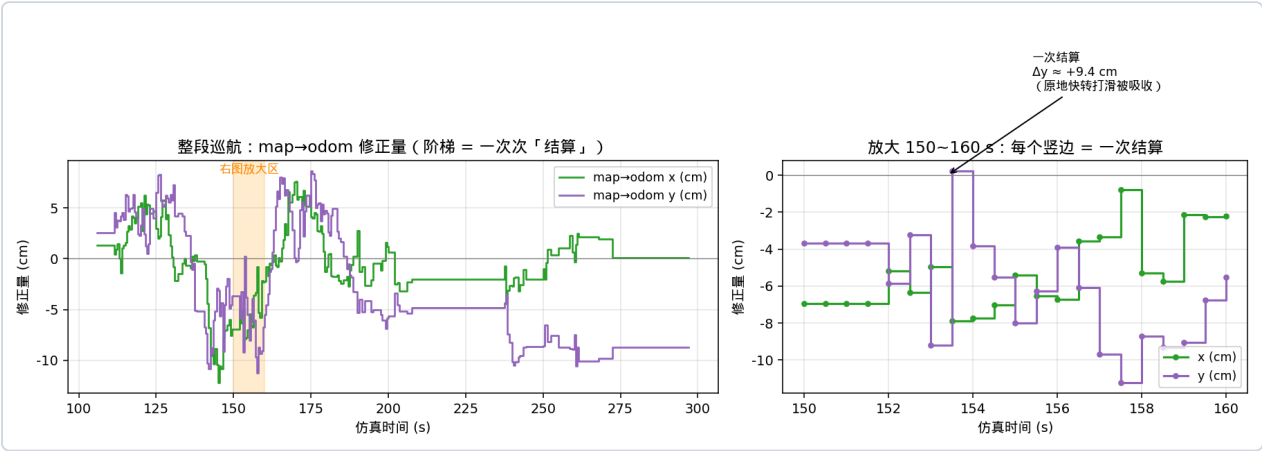
右图（修正量随时间）：绿色是 map → odom 平移修正量（cm），紫色是角度修正量（°）。三个读图要点：

1. **量级：**整圈修正量在 $0.4 \pm 2.6 \text{ cm}$ 、 $-2^\circ \pm 3.7^\circ$ 之间。对一张 0.05 m 分辨率的地图来说，这个漂移不矫正的话，走完一圈误差就够把机器人"定位"到墙里了。
2. **阶梯状：**曲线不是连续变化而是一格一格跳——AMCL 只在车移动 0.15 m 或转 0.15 rad 后才做一次滤波更新（update_min_d/update_min_a），每次更新修正量跳一格，其间保持常数。这个形态的设计原因见 4.5 节。
3. **运动激烈处修正大：**060 s（出发、转弯密集）修正量波动最大；80~140 s 直行段修正量几乎不变——车不打滑，里程计就不添新账。

指标	实测值
采样时长 / 点数	157 s (仿真时间) / 315 点
map → odom 平移修正	最小 0.4 cm, 最大 12.6 cm
map → odom 角度修正	最大 3.7°
更新形态	阶梯状 (每 0.15 m / 0.15 rad 结算一次)

5.1 显微镜下的一次结算（实测数据逐帧核对 4.1 的四步）

上面看的是整圈全貌，现在把镜头推到**单独一次结算**上。我们重录了一组数据（同样是 Gazebo 巡航，record_drift.py 每 0.5 s 采一个点），先看曲线：



单次结算放大

左图是整段巡航的 map → odom 修正量，右图放大 150~160 s。可以清楚看到：曲线不是斜坡而是一格一格的**台阶**，每个竖边就是一次结算。其中 153.5 s 附近有一次特别大的竖边——y 方向修正量一步跳了约 +9.4 cm。为什么恰好是这里？因为此刻机器人正在巡逻点**原地快速转向**（0.5 s 内偏航角从 57.7° 转到 86.9°），而原地自转是麦轮打滑最狠的动作（§ 2 表格），里程计在这半秒里漂出了一大笔新账，被这次结算一口吸收。

把这次结算前后的原始记录摆出来，逐项对上 4.1 节的四步（单位 m / 度）：

时刻	① 里程计位姿 $T_{\text{odom} \rightarrow \text{base}}$	② 激光估计 $T_{\text{map} \rightarrow \text{base}}$	③④ 发布的 $T_{\text{map} \rightarrow \text{odom}}$
153.0 s	$(2.527, 1.224, 57.7^\circ)$	$(2.450, 1.188, 59.0^\circ)$	$(-0.049, -0.092, +1.25^\circ)$
153.5 s	$(2.456, 1.426, 86.9^\circ)$	$(2.395, 1.399, 86.2^\circ)$	$(-0.079, +0.002, -0.69^\circ)$

三点核对：

- 每一行都满足 4.1 的公式：② = ③ ◦ ①（把 map → odom 作用在里程计位姿上，恰好得到激光估计的 map 系位姿）。
- 两行之间 ③ 的变化（y 跳 +9.4 cm、角度跳约 -1.9°）就是右图那个竖边——**这半秒原地快转新漂出来的量**。

3. ① 没有被任何人改动，它继续按自己的节奏走；变的只有账本 ③。

同一件事在 `tf2_echo` 终端里长这样（实录，节选自 `/tmp/tf_echo_settle.txt`，两次采样跨过了这次结算）：

```
At time 153.800000000
- Translation: [-0.058, -0.080, 0.000]
- Rotation: in RPY (degree) [0.000, -0.000, 1.075]
At time 154.100000000
- Translation: [-0.075, -0.012, 0.000]
- Rotation: in RPY (degree) [0.000, 0.000, -0.592]
```

0.3 s 之间 Translation 的 y 从 -0.080 跳到 -0.012——你在第 6 节 LAB 里盯着终端看到的"跳一下"，就是这个。

6. LAB：亲手观察漂移矫正（10 分钟）

启动导航栈（两个后端均可，Gazebo 漂移更真实）：

```
cd ~/mecamind_ros2
bash scripts/cleanup_ros2.sh --all
source /opt/ros/jazzy/setup.bash && source install/setup.bash
ros2 launch mecamind_bringup navigation.launch.py
```

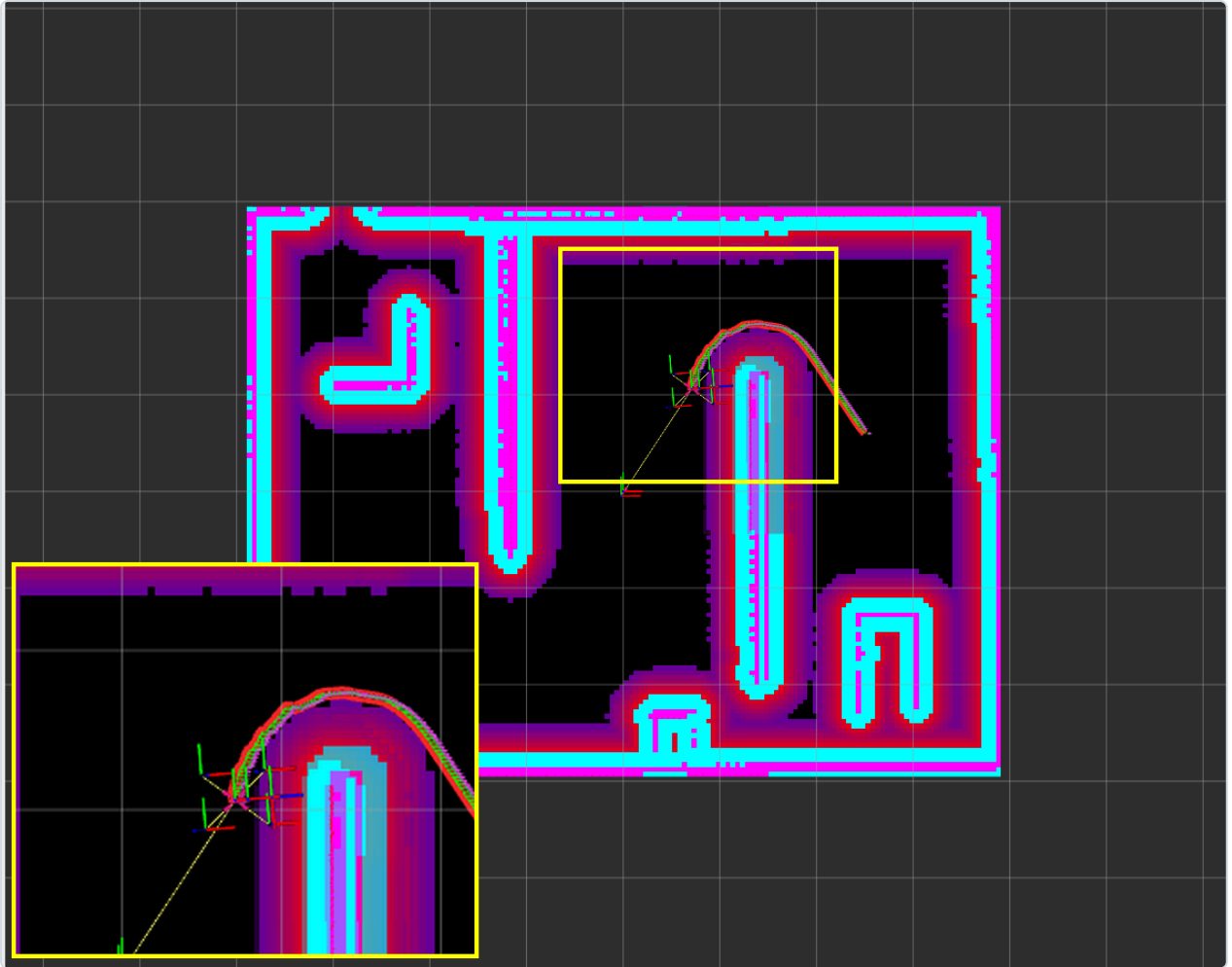
6.1 直接看修正量

```
# map→odom 就是漂移修正量：静止时不变，运动后每隔一小段"跳"一次
ros2 run tf2_ros tf2_echo map odom --ros-args -p use_sim_time:=true
```

发个巡航任务让车动起来，盯着 Translation 的变化：

```
ros2 topic pub --once /mecamind/task_command std_msgs/msg/String \
  '{data: "{\\"intent\\": \\"patrol\\"}"}'
```

巡航中的 RViz 长这样（实测截图，左下角是机器人附近的放大图）：彩色渐变是代价地图（青色 = 障碍膨胀后的禁区），从机器人延伸出去的曲线是全局路径，机器人本体位置的坐标轴就是 TF 树的 `base_footprint`。AMCL 的粒子云紧紧收缩成机器人脚下的一小簇——粒子抱团越紧说明定位越自信，它们的加权均值就是 4.1 节公式里的 $T_{\text{map} \rightarrow \text{base}}$ ：



巡航中的 RViz：全局路径与收敛的粒子云

6.2 录制并出图

```
# 巡航开始后另开终端录制，Ctrl+C 结束（CSV 边采边写，随时可停）  
python3 scripts/record_drift.py /tmp/drift.csv
```

CSV 各列：`t`（仿真时间）、`mo_*`（map→odom 平移与偏航）、`ob_*`（odom→base_footprint）。合成 map 系位姿的公式在第 3 节——两段 TF 相乘。

6.3 在 RViz 里看"跳变"

RViz 中把 Fixed Frame 临时切到 `odom`：静态地图会随每次 AMCL 更新整体"挪动"一小步——地图在 odom 系里跳，正是 `map → odom` 被改写的可视化。看完记得切回 `map`。

7. 常见问题

现象	原因	处置
map → odom 恒为 0 且不变	定位没在工作（AMCL 未激活 / 没给初始位姿）	查 lifecycle 状态；RViz 2D Pose Estimate 重新初始化
修正量突然跳到几米	定位丢失后错误收敛（初值给错房间、长走廊特征退化）	只能重新给初始位姿；粒子滤波是局部收敛的，拉不回来
修正量持续单调增大	里程计系统性偏差（轮径标定错、一侧轮胎异常）	修里程计本身，矫正层只能兜住随机漂移，兜不住系统性错误
控制器抖动、车走 S 形	有节点误用了 map 系做控制	控制回路一律用 odom 系（局部代价地图挂 odom 系就是为此）